UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
INF05010 – OTIMIZAÇÃO COMBINATÓRIA

Proposta de Implementação

Meta-heurística Simulated Annealing (SimAnn) para o Problema do Jardim Imperial das Flores Caprichosas (JIFC)

Alunos: Antônio Carlos Gonçalves Sarti – (00130172),
Giovanni Rosell Paredes dos Santos – (00343890)

Professor: Prof. Dr. Marcus Ritt

Porto Alegre
2026

1 Resumo Formal e Modelo Matemático do JIFC

O problema do **Jardim Imperial das Flores Caprichosas (JIFC)** consiste em selecionar um subconjunto de setores a serem cultivados em um jardim representado por um grafo não-direcionado, visando maximizar o esplendor total sob restrições de conectividade e capacidade hídrica.

1.1 Definição Formal da Instância

Uma instância do JIFC é representada pela tupla $I = (G, p, e, a, f, H)$, onde:

- $G = (S, E)$ é um grafo não-direcionado simples e conexo, no qual $S = [n] = \{1, 2, \ldots, n\}$ representa o conjunto de setores e E representa o conjunto de portões (arestas);
- $p \in S$ denota o setor onde está localizado o Palácio Imperial;
- $e_i \geq 0$ é o esplendor proporcionado pelo cultivo do setor $i \in S$;
- $a_i \geq 0$ é o consumo normal de água do setor $i \in S$;
- $f_i \geq 0$ é o consumo adicional de água do setor $i \in S$;
- H é a capacidade total do aqueduto imperial.

1.2 Restrições do Problema

Uma solução válida é um subconjunto de setores cultivados $C \subseteq S$ que satisfaz:

1. **Presença obrigatória do Palácio:** $p \in C$.
2. **Conectividade:** o subgrafo induzido $G[C]$ deve ser conexo. Como $p \in C$, todos os setores cultivados devem ser alcançáveis a partir do Palácio utilizando somente setores pertencentes a C.
3. **Restrição hídrica:** para todo $F \subseteq C$ com $|F| \leq 2$,

$$\sum_{i \in C} a_i + \sum_{i \in F} f_i \leq H.$$

Como o pior caso corresponde à escolha dos dois maiores valores de f_i entre os setores cultivados, a restrição pode ser escrita como:

$$W(C) = \sum_{i \in C} a_i + \text{top}_2(\{f_i : i \in C\}) \leq H. \tag{1}$$

Para $|C| = 1$, considera-se apenas o maior valor disponível, enquanto, para $|C| \geq 2$, top_2 corresponde à soma dos dois maiores valores de f_i pertencentes a C.

Assim, uma solução é viável quando contém o Palácio, é conexa e satisfaz $W(C) \leq H$.

1.3 Função Objetivo

O objetivo é maximizar o esplendor total dos setores cultivados:

$$\max \text{esplendor}(C) = \sum_{i \in C} e_i. \tag{2}$$

1.4 Formulação Matemática do Problema

O JIFC pode ser formulado como um problema de otimização combinatória de maximização, no qual se busca selecionar um subconjunto de setores cultivados que maximize o esplendor total, respeitando as restrições de presença do Palácio Imperial, conectividade e capacidade hídrica.

1.4.1 Variáveis de Decisão

Para cada setor $i \in S$, define-se a variável binária:

$$x_i = \begin{cases} 1, & \text{se o setor } i \text{ é cultivado,} \\ 0, & \text{caso contrário.} \end{cases} \tag{3}$$

O conjunto de setores selecionados é dado por:

$$C = \{i \in S \mid x_i = 1\}. \tag{4}$$

1.4.2 Modelo de Otimização

A formulação matemática do problema é:

$$\boxed{\begin{aligned} \max \quad & Z = \sum_{i \in S} e_i x_i \\ \text{sujeito a:} \quad & x_p = 1, \\ & G[C] \text{ é conexo}, \\ & \sum_{i \in C} a_i + \text{top}_2(\{f_i : i \in C\}) \leq H, \\ & C = \{i \in S : x_i = 1\}, \\ & x_i \in \{0, 1\}, \quad \forall i \in S. \end{aligned}} \tag{5}$$

A função objetivo maximiza o esplendor total dos setores cultivados. A primeira restrição garante a inclusão obrigatória do Palácio Imperial. A segunda exige que todos os setores selecionados estejam conectados ao Palácio por caminhos formados exclusivamente por setores cultivados.

A terceira restrição assegura que o consumo hídrico não ultrapasse a capacidade H, mesmo quando até dois setores cultivados apresentam simultaneamente consumo adicional. Por fim, as variáveis binárias determinam quais setores pertencem à solução.

1.4.3 Relação com a Meta-heurística

A formulação apresentada define o conjunto de soluções viáveis e a função objetivo do problema, independentemente do método de otimização empregado.

Na implementação proposta, o Simulated Annealing explora esse conjunto por meio dos operadores *Add*, *Remove* e *Swap*, considerando exclusivamente movimentos que preservem todas as restrições do modelo.

Para uma solução atual C e uma solução vizinha viável C', define-se a variação do esplendor como:

$$\Delta = \text{esplendor}(C') - \text{esplendor}(C). \tag{6}$$

A probabilidade de aceitação da solução vizinha, para uma temperatura $T > 0$, é dada por:

$$P(C \to C') = \begin{cases} 1, & \text{se } \Delta \geq 0, \\ \exp\left(\dfrac{\Delta}{T}\right), & \text{se } \Delta < 0. \end{cases} \tag{7}$$

Assim, movimentos que melhoram ou mantêm o esplendor são aceitos, enquanto movimentos de piora podem ser aceitos probabilisticamente, permitindo a exploração de diferentes regiões do espaço de soluções.

Ressalta-se que a formulação apresentada constitui um modelo de otimização combinatória, não uma formulação linear inteira completa, uma vez que a conectividade é expressa como propriedade do subgrafo induzido, sem sua linearização explícita.

2 Formulação do Simulated Annealing (SimAnn)

O Simulated Annealing será utilizado para explorar diferentes soluções do JIFC por meio de movimentos de vizinhança. A cada iteração, uma solução vizinha da solução atual é gerada e avaliada. Soluções que melhoram o valor da função objetivo são aceitas, enquanto soluções piores podem ser aceitas com uma probabilidade que diminui conforme a temperatura do algoritmo é reduzida.

Nesta abordagem, serão utilizadas somente soluções viáveis. Dessa forma, cada solução considerada pelo algoritmo deve respeitar as restrições de presença do Palácio, conectividade e capacidade hídrica.

2.1 Representação da Solução e Estruturas Auxiliares

Uma solução é representada pelo conjunto de setores cultivados $C \subseteq S$, com $p \in C$.

Para facilitar a implementação, será utilizado um vetor booleano `in_solution`, no qual:

- `in_solution[i] = true` indica que o setor i pertence a C;
- `in_solution[i] = false` indica que o setor i não pertence a C.

Também serão mantidos:

- o esplendor corrente $E(C) = \sum_{i \in C} e_i$;
- o consumo normal $A(C) = \sum_{i \in C} a_i$;
- os dois maiores valores de f_i entre os setores pertencentes a C.

Dessa forma, o valor de $W(C)$ pode ser obtido em tempo constante a partir das estruturas mantidas.

Após uma inclusão, os dois maiores valores de f_i podem ser atualizados comparando o novo setor com os valores atualmente armazenados. Após uma remoção, entretanto, um dos dois maiores valores pode deixar de pertencer à solução. Nesse caso, os dois maiores valores serão recomputados percorrendo os setores de C.

Essa escolha privilegia a simplicidade e a correção da implementação, evitando a necessidade de uma estrutura de dados mais complexa.

A conectividade será verificada por uma busca em largura (BFS) ou busca em profundidade (DFS) sobre $G[C]$ sempre que necessário. Como a representação utiliza o vetor booleano, a busca considera apenas setores atualmente pertencentes à solução.

3 Geração da Solução Inicial

A solução inicial será construída de forma gulosa, iniciando pelo Palácio Imperial:

$$C = \{p\}.$$

Antes da construção, verifica-se se $a_p + f_p \leq H$. Caso essa condição não seja satisfeita, a instância é inviável, pois o Palácio deve pertencer a toda solução e os consumos são não negativos.

A partir dessa solução, são considerados setores adjacentes a C que possam ser adicionados sem violar a capacidade hídrica.

Para cada candidato $u \notin C$, calcula-se o incremento exato do consumo hídrico decorrente de sua inclusão:

$$\Delta W(u \mid C) = W(C \cup \{u\}) - W(C). \tag{8}$$

O ganho de esplendor por unidade de consumo adicional será então calculado por:

$$r(u) = \frac{e_u}{\Delta W(u \mid C) + \epsilon}, \tag{9}$$

onde $\epsilon > 0$ é uma pequena constante utilizada para evitar divisão por zero. Em cada etapa, são considerados apenas os candidatos cuja inclusão mantém a solução viável. Entre esses candidatos, seleciona-se aquele que apresenta a maior razão $r(u)$ e adiciona-se o setor à solução. O procedimento termina quando não houver mais setores adjacentes a C cuja inclusão mantenha a solução viável. Por construção, a solução inicial contém o Palácio, permanece conexa e é viável.

4 Operadores de Vizinhança

Serão utilizados três operadores básicos de vizinhança: *Add*, *Remove* e *Swap*. Em cada iteração, um dos três operadores será selecionado aleatoriamente, com probabilidades de 40%, 30% e 30%, respectivamente. Após a seleção do operador, serão considerados somente os movimentos que resultem em soluções viáveis. Um dos movimentos viáveis do operador selecionado será escolhido uniformemente. Caso o operador selecionado não possua nenhum movimento viável na solução atual, a iteração será encerrada sem alteração da solução. Nesse caso, não será realizado um novo sorteio de operador. Essa regra preserva as probabilidades nominais de seleção de 40% para *Add*, 30% para *Remove* e 30% para *Swap*. Se nenhum operador possuir movimento viável, a solução atual também será mantida. Em ambos os casos, a iteração será contabilizada normalmente no tamanho do patamar de temperatura L.

Table 1: Operadores de vizinhança.

Operador	**Probabilidade**	**Descrição Operacional**
Add (Inclusão)	40%	Considera os setores $u \notin C$ que possuem pelo menos um vizinho em C. Um candidato é válido quando $C' = C \cup \{u\}$ permanece conexo e satisfaz $W(C') \leq H$. Um dos candidatos válidos é sorteado uniformemente.
Remove (Exclusão)	30%	Considera os setores $u \in C \setminus \{p\}$. Um candidato é válido quando $C' = C \setminus \{u\}$ permanece conexo e satisfaz $W(C') \leq H$. Um dos candidatos válidos é sorteado uniformemente.
Swap (Troca)	30%	Considera pares (u, v), com $u \in C \setminus \{p\}$ e $v \notin C$. Um par é válido quando $C' = (C \setminus \{u\}) \cup \{v\}$ permanece conexo e satisfaz $W(C') \leq H$. Um dos pares válidos é sorteado uniformemente.

O operador *Add* permite aumentar o número de setores cultivados. O operador *Remove* permite liberar capacidade hídrica para possibilitar outras combinações. O operador *Swap* combina as duas operações e permite substituir setores de menor interesse por outros setores.

Embora a solução atual seja sempre viável, a geração dos candidatos dos operadores será realizada verificando explicitamente as restrições, garantindo que somente vizinhos válidos sejam considerados.

5 Avaliação das Soluções e Custo Computacional

O valor da função objetivo de uma solução será mantido incrementalmente por meio de $E(C)$. Assim, uma inclusão, remoção ou troca permite atualizar o esplendor em tempo constante. O consumo normal $A(C)$ também será mantido incrementalmente. As alterações decorrentes de inclusão e remoção são, portanto, realizadas em tempo constante. Os dois maiores valores de f_i serão mantidos explicitamente. Uma inclusão pode atualizar esses valores em tempo constante. Em uma remoção, caso um dos dois maiores valores seja removido, os dois maiores valores serão recomputados percorrendo C, com custo $O(|C|)$. A verificação da conectividade será realizada

por BFS ou DFS no subgrafo induzido $G[C]$, com custo

$$O(|S| + |E|).$$

Consequentemente, a verificação de um candidato de *Add*, *Remove* ou *Swap* terá custo dominado pela verificação de conectividade, sendo $O(|S| + |E|)$, além do custo de atualização ou recomputação dos dois maiores valores de f_i quando necessário. Para *Swap*, a avaliação dos pares candidatos pode exigir a verificação de diversos pares. Nesse caso, a construção da vizinhança tem custo proporcional ao número de pares candidatos avaliados, multiplicado pelo custo de verificação de viabilidade de cada par. No operador *Swap*, existem no máximo $|C \setminus \{p\}|\,(|S| - |C|) = O(|C|(|S| - |C|))$ pares candidatos. Se todos forem testados por BFS ou DFS, o limite superior do custo de geração dessa vizinhança é $O(|C|(|S| - |C|)(|S| + |E|))$. Essa estratégia prioriza uma implementação simples e correta. A avaliação de possíveis otimizações dessas estruturas auxiliares poderá ser realizada posteriormente, caso os experimentos indiquem que o custo de avaliação constitui o principal gargalo computacional.

6 Esquema de Resfriamento e Parâmetros

O Simulated Annealing utilizará um esquema de resfriamento geométrico.

6.1 Calibração da Temperatura Inicial

A calibração utiliza uma caminhada exploratória independente da execução principal, iniciada na solução gulosa C_0. Em cada tentativa, sorteia-se um operador com as mesmas probabilidades da Seção 4 e, quando houver vizinhos viáveis, seleciona-se um deles uniformemente. Registra-se $\Delta = \text{esplendor}(C') - \text{esplendor}(C)$ e atualiza-se a solução exploratória para C', permitindo observar também movimentos de remoção e de piora. A caminhada não altera C_{atual} nem C_{best} da execução principal.

Serão realizadas inicialmente $N_{\text{cal}} = 100$ tentativas. Caso não seja observado nenhum $\Delta < 0$, a amostragem prosseguirá até um limite total de $N_{\max} = 10N_{\text{cal}} = 1000$ tentativas. Tentativas sem vizinho viável contam para esse limite, mas não geram valores de Δ.

Define-se a escala positiva d pela seguinte regra, aplicada aos movimentos efetivamente observados:

$$d = \begin{cases} \text{media}\{|\Delta| : \Delta < 0\}, & \text{se houver movimentos de piora,} \\ \text{media}\{|\Delta| : \Delta \neq 0\}, & \text{se não houver pioras, mas houver variações não nulas,} \\ 1, & \text{se todas as variações observadas forem nulas ou inexistentes.} \end{cases} \tag{10}$$

A última alternativa é uma convenção de segurança, expressa nas unidades do esplendor, que evita temperaturas indefinidas. Quando não há movimentos viáveis na solução inicial, a execução pode ser encerrada imediatamente, retornando essa solução, pois não existe movimento permitido para iniciar a busca. Nos demais casos, o resfriamento prossegue mesmo quando a escala foi estimada apenas por variações positivas ou pela convenção $d = 1$.

Adotando a probabilidade-alvo inicial $p_0 = 0.8$, obtém-se:

$$T_0 = -\frac{d}{\ln(p_0)}. \tag{11}$$

Assim, sempre que existirem pioras amostradas, a escala coincide com a média de suas magnitudes; as demais alternativas são regras explícitas para os casos excepcionais.

6.2 Resfriamento

Será utilizado resfriamento geométrico:

$$T_{k+1} = \alpha T_k,$$

com configuração inicial

$$\alpha = 0.95.$$

6.3 Tamanho do Patamar

Para cada nível de temperatura serão realizadas

$$L = 50|S|$$

iterações. Uma iteração na qual o operador selecionado não possui movimento viável será contabilizada normalmente no patamar de temperatura, sem alteração da solução atual.

6.4 Critério de Aceitação

Seja

$$\Delta = \text{esplendor}(C_{\text{viz}}) - \text{esplendor}(C_{\text{atual}}).$$

Se $\Delta > 0$, o vizinho será aceito. Caso contrário, será aceito com probabilidade

$$P = \exp\left(\frac{\Delta}{T}\right).$$

A decisão será realizada comparando P com um número aleatório uniformemente distribuído em $[0, 1)$.

6.5 Temperatura Mínima e Critério de Parada

O limite mínimo de temperatura será definido a partir da mesma escala positiva d utilizada na calibração de T_0. Será adotada uma probabilidade final alvo

$$p_{\min} = 10^{-3}$$

para uma piora de magnitude d. Assim,

$$T_{\min} = -\frac{d}{\ln(p_{\min})}. \tag{12}$$

Dessa forma, o critério de temperatura mínima não depende de uma escala absoluta arbitrária e permanece relacionado à magnitude das variações da função objetivo.

Além da temperatura mínima, será utilizado um limite máximo de níveis de temperatura, definido inicialmente por

$$K_{\max} = 100.$$

Como cada nível realiza $L = 50|S|$ iterações, o número máximo de iterações será

$$I_{\max} = K_{\max} \cdot L = 5000|S|.$$

A execução será encerrada quando ocorrer uma das seguintes condições:

1. $T \leq T_{\min}$; ou
2. for atingido o nível máximo de temperatura $K_{\max}$.

Assim, o critério de parada utilizado no algoritmo e a definição dos parâmetros permanecem consistentes.

7 Resultados de Referência

Os melhores valores conhecidos (*Best Known Values* – BKV) serão utilizados como referência para avaliar a qualidade das soluções obtidas pelo Simulated Annealing.

Table 2: Melhores valores conhecidos (BKV) para as instâncias do JIFC.

Instância	**BKV (Esplendor)**	**Instância**	**BKV (Esplendor)**
jifc01	88063	jifc07	118878
jifc02	136257	jifc08	141621
jifc03	186418	jifc09	45345
jifc04	209722	jifc10	59368
jifc05	62273	jifc11	76498
jifc06	103748	jifc12	88180

8 Algoritmo Principal

Algoritmo 1: Simulated Annealing para o JIFC

Entrada: Instância I; $\alpha = 0.95$; $L = 50|S|$; $N_{\text{cal}} = 100$; $N_{\max} = 10N_{\text{cal}}$; $p_0 = 0.8$; $p_{\min} = 10^{-3}$; $K_{\max} = 100$

Saída: Melhor solução encontrada C_{best}

if $a_p + f_p > H$ **then**
  retorna *instância inviável*

$C_{\text{atual}} \leftarrow \text{GerarSolucaoInicial}(I)$
$C_{\text{best}} \leftarrow C_{\text{atual}}$
if *não existe vizinho viável de* C_{atual} **then**
  retorna C_{best}

$d \leftarrow \text{CalibrarEscala}(C_{\text{atual}}, N_{\text{cal}}, 10N_{\text{cal}})$
$T_0 \leftarrow -d/\ln(p_0)$
$T_{\min} \leftarrow -d/\ln(p_{\min})$
$T \leftarrow T_0$
$k \leftarrow 0$
while $T > T_{\min}$ ***e*** $k < K_{\max}$ **do**
  for *iter* $\leftarrow 1$ **to** L **do**
    $C_{\text{viz}} \leftarrow \text{GerarVizinho}(C_{\text{atual}})$
    if C_{viz} *existe* **then**
      $\Delta \leftarrow \text{esplendor}(C_{\text{viz}}) - \text{esplendor}(C_{\text{atual}})$
      if $\Delta \geq 0$ ***ou*** $random(0,1) < \exp(\Delta/T)$ **then**
        $C_{\text{atual}} \leftarrow C_{\text{viz}}$
        if $esplendor(C_{atual}) > esplendor(C_{best})$ **then**
          $C_{\text{best}} \leftarrow C_{\text{atual}}$
  $T \leftarrow \alpha T$
  $k \leftarrow k + 1$
retorna C_{best}

Algoritmo 2: Calibração limitada da escala de temperatura

Entrada: Solução viável C_0; N_{cal}; $N_{\max}$

Saída: Escala positiva d

$C_{\text{exp}} \leftarrow C_0$; $D \leftarrow \emptyset$; $t \leftarrow 0$

while $t < N_{\max}$ ***e*** *(*$t < N_{\text{cal}}$ ***ou*** *não há* $\Delta < 0$ *em* D*)* **do**

- $t \leftarrow t + 1$
- $C' \leftarrow \text{GerarVizinho}(C_{\text{exp}})$
- **if** C' *existe* **then**
  - $\Delta \leftarrow \text{esplendor}(C') - \text{esplendor}(C_{\text{exp}})$
  - $D \leftarrow D \cup \{\Delta\}$; $C_{\text{exp}} \leftarrow C'$

if *existe* $\Delta < 0$ *em* D **then**

- $d \leftarrow \text{media}\{|\Delta| : \Delta \in D,\ \Delta < 0\}$

else

- **if** *existe* $\Delta \neq 0$ *em* D **then**
  - $d \leftarrow \text{media}\{|\Delta| : \Delta \in D,\ \Delta \neq 0\}$
- **else**
  - $d \leftarrow 1$

retorna d